Day 16 Task : Docker Project for DevOps Engineers.

Maninder Singh

Dear Learners in today topic we will learn the Deploy Python-Django web app to AWS EC2 with Docker.

Docker file:

Docker is a tool that makes it easy to run applications in containers. Containers are like small packages. To create the containers, developers use something called a Docker file.

Docker file is a script that contains a set of instructions to build a docker image. It is a lightweight, standalone, and executable.

Task:

A) Create a Docker file for a simple web application (for Ex- Node.js)

B) Build the image using the docker file and run the container

C) verify that the application is working as expected by accessing it in a web browser.

D) Push the image to a public or private repository (for ex-Docker hub)

Steps to create:-

1 Launch AWS ubuntu ec2 instance.

2 Clone the repository form Github to your server using commands.

Syntax: git clone <repository_url>

```
ubuntu@ip-172-31-40-211:~$ mkdir python
ubuntu@ip-172-31-40-211:~$ ls
 90                                    code.txt    kodekloud.txt      python                  version01.txt
```

Maninder Singh

```
 90-days-of-devops                                  day13.py    linux.txt          react_django_demo_app
 Devops                                             day14.py    new.txt            test.txt
'There is a new file written in the kodekloud'      git01.txt   nginx-backup.tar   test.txt.save
ubuntu@ip-172-31-40-211:~$ cd python/
ubuntu@ip-172-31-40-211:~/python$ ls
ubuntu@ip-172-31-40-211:~/python$ git clone https://github.com/LondheShubham153/react_django_demo_app.git
Cloning into 'react_django_demo_app'...
remote: Enumerating objects: 167, done.
remote: Counting objects: 100% (33/33), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 167 (delta 21), reused 14 (delta 11), pack-reused 134
Receiving objects: 100% (167/167), 513.61 KiB | 14.27 MiB/s, done.
Resolving deltas: 100% (41/41), done.
ubuntu@ip-172-31-40-211:~/python$ ls
react_django_demo_app
ubuntu@ip-172-31-40-211:~/python$ cd react_django_demo_app/
ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$ ls
Dockerfile  README.md  api  db.sqlite3  docker-compose.yaml  frontend  manage.py  requirements.txt  tests  todo_drf
ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$
```

Git clone and enter in the react folder Diagram.

3 Create a Docker file.

```
ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$ ls
Dockerfile  README.md  api  db.sqlite3  docker-compose.yaml  frontend  manage.py  requirements.txt  tests  todo_drf
ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$ vi Dockerfile
ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$ cat Dockerfile
FROM python:3.9
WORKDIR app
COPY . /app
RUN pip install -r requirements.txt
EXPOSE 8001
CMD ["python","manage.py","runserver","0.0.0.0:8001"]

ubuntu@ip-172-31-40-211:~/python/react_django_demo_app$
```

Docker File create Diagram (Dockerfile)

Maninder Singh

Now lets us discuss the docker file points:-

FROM python:3.9:

it is used to which image we want to build from. I am using the python 3.9

WORKDiR /app

we create a directory to hold the application code inside the images.

COPY . /app

copy your application code inside the docker image.

RUN pip install -r requirement.txt

Command used for the all dependencies in the requirement file.

EXPOSE 8000

commands used for release port 8000 within the container.

CMD ["python","manage.py","runserver","0.0.0.0:8001"]

Command starts the server and runs the application.

4 Build an image using the docker file.

Syntax: docker build -t <image>

Maninder Singh

```
Build an image from a Dockerfile
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker build -t django-app-img:v1 .
Sending build context to Docker daemon  2.667MB
Step 1/6 : FROM python:3.9
3.9: Pulling from library/python
b0248cf3e63c: Pull complete
127e97b4daf7: Pull complete
0336c50c9f69: Pull complete
1b89f3c7f7da: Pull complete
2d6277217976: Pull complete
273fcda609d8: Pull complete
10e914379794: Pull complete
```

```
380814be3a80: Pull complete
23224c967057: Pull complete
Digest: sha256:2d8875d28ca023a9056a828518adcdd634ba03740e1e3b197c06eb4527c6152c
Status: Downloaded newer image for python:3.9
 ---> 2202efa5d0ba
Step 2/6 : WORKDIR app
 ---> Running in d24067754340
Removing intermediate container d24067754340
 ---> 18412b175a77
Step 3/6 : COPY . /app
 ---> 5517a1b7c343
Step 4/6 : RUN pip install -r requirements.txt
 ---> Running in 225015e3a255
Collecting asgiref==3.2.3
  Downloading asgiref-3.2.3-py2.py3-none-any.whl (18 kB)
Collecting Django==3.0.3
  Downloading Django-3.0.3-py3-none-any.whl (7.5 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.5/7.5 MB 52.0 MB/s eta 0:00:00
Collecting django-cors-headers==3.2.1
  Downloading django_cors_headers-3.2.1-py3-none-any.whl (14 kB)
Collecting djangorestframework==3.11.0
  Downloading djangorestframework-3.11.0-py3-none-any.whl (911 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 911.2/911.2 KB 49.6 MB/s eta 0:00:00
Collecting pytz==2019.3
```

sudo docker build -t django-app-img:v1 . Diagram

using the docker images command we can see the list of images.

syntax: sudo docker images

sudo docker images Diagram.

5 Run the image to create a container.

syntax: sudo docker run -d --name <container_name> -p 8000:8000 <iamge-name>

docker ps (command used for the list of all the containers)

```
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker run -d --name django-demo-app -p 8000:8000 django-app-img:v1
6e9b18e8386ce04cbbdb8e6b932c16ea8cefde773748771b45917aaaf99af63d
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                                                   NAMES
6e9b18e8386c   django-app-img:v1  "python manage.py ru…"   7 seconds ago   Up 6 seconds   0.0.0.0:8000->8000/tcp, :::8000->8000/tcp, 8001/tcp     django-demo-app
2b1130b7e19d   nginx:alpine       "/docker-entrypoint.…"   26 hours ago    Up 26 hours    0.0.0.0:8081->80/tcp, :::8081->80/tcp                   nginx-new-container
a771fb995222   nginx:alpine       "/docker-entrypoint.…"   26 hours ago    Up 26 hours    0.0.0.0:8080->80/tcp, :::8080->80/tcp                   sleepy_wright
da6b3a3abb87   nginx:alpine       "/docker-entrypoint.…"   28 hours ago    Up 28 hours    80/tcp                                                  nginx-container
deecb80ad698   nginx:alpine       "/docker-entrypoint.…"   28 hours ago    Up 28 hours    80/tcp                                                  unruffled_yalow
ubuntu@ip-172-31-40-211:~/react_django_demo_app$
```

Manindar Singh

5 Run the image to create a container. Diagram.

6 Now team at the last verify the application is working check it in a web browser with port no

Syntax: http://ec2-instance-public-ip:8000

7 Push the image to a public or private repository for ex docker hub

```
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker run -d --name django-demo-app -p 8000:8000 django-app-img:v1
6e9b18e8386ce04cbbdb8e6b932c16ea8cefde773748771b45917aaaf99af63d
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                                                   NAMES
6e9b18e8386c   django-app-img:v1  "python manage.py ru…"   7 seconds ago   Up 6 seconds   0.0.0.0:8000->8000/tcp, :::8000->8000/tcp, 8001/tcp     django-demo-app
2b1130b7e19d   nginx:alpine       "/docker-entrypoint.…"   26 hours ago    Up 26 hours    0.0.0.0:8081->80/tcp, :::8081->80/tcp                   nginx-new-container
a771fb995222   nginx:alpine       "/docker-entrypoint.…"   26 hours ago    Up 26 hours    0.0.0.0:8080->80/tcp, :::8080->80/tcp                   sleepy_wright
da6b3a3abb87   nginx:alpine       "/docker-entrypoint.…"   28 hours ago    Up 28 hours    80/tcp                                                  nginx-container
deecb80ad698   nginx:alpine       "/docker-entrypoint.…"   28 hours ago    Up 28 hours    80/tcp                                                  unruffled_yalow
ubuntu@ip-172-31-40-211:~/react_django_demo_app$
```

docker ps commads diagram

```
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com to create one.
Username: manindersingh
Password:
Error response from daemon: Get "https://registry-1.docker.io/v2/": unauthorized: incorrect username or password
ubuntu@ip-172-31-40-211:~/react_django_demo_app$ sudo docker login
Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com to create one.
Username: dockerps123
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

docker login with user name and password

Push docker image to docker hub command

syntax: docker push <image-name>

Thank you for reading !! I hope you find this article helpful!!

Happy Learning!!

Manindder Singh

Next Topic

Day 17 Task: Docker for DevOps Engineers -Part2!!

Manindar Singh